

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO

Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de Semicondutores

Carlos Oliveira da Silva
Emanuelle Lima e Lima
Gabriel Gomes Jacinto

SISTEMA DIGITAL FUNCIONAL

BANCO DE REGISTRADORES

03/09/2026
MANAUS - AM

Objetivo

Consolidar o exercício anterior integrando os módulos individuais (latches e flipflops) desenvolvidos por cada integrante em um sistema digital funcional, introduzindo conceitos de projeto hierárquico, interface entre blocos, verificação de sistemas e (opcionalmente) síntese lógica.

Enunciado proposto ao grupo

O grupo deve escolher um dos sistemas abaixo para projetar, utilizando como base os módulos de memória que cada membro já implementou na etapa anterior (cada aluno contribui com seu elemento como submódulo):

1. Banco de registradores (Register File) — N registradores de M bits, com decodificação de endereço de leitura/escrita, usando FFs-D como célula de armazenamento.

2. Registrador de deslocamento (shift register) serial-paralelo, com modos de carga paralela e deslocamento à esquerda/direita.

3. Contador Johnson ou anel de deslocamento usando FFs-T ou FFs-JK.

4. Memória RAM simples (ex. 4x4 ou 8x4 bits) usando latches D como células, com lógica de endereçamento e controle de escrita/leitura.

5. Máquina de estados finita (FSM) aplicada (ex. controlador de semáforo ou máquina de venda automática), usando os flip-flops do grupo como elementos de estado.

Requisitos mínimos (nível acima da etapa individual)

1. Arquitetura hierárquica: o sistema deve instanciar os módulos individuais já validados como submódulos, não recomeçar do zero.

2. Especificação de interface acordada em grupo antes de codificar: largura de barramento, sinais de controle, convenção de endereçamento, protocolo de clock/reset.

3. Testbench de integração com verificação automática (self-checking): em vez de só observar a waveform, o testbench deve comparar a saída simulada com um valor esperado e sinalizar PASS/FAIL (assertions do SystemVerilog, se o grupo quiser ir além).

4. Análise temporal do sistema completo: identificar o caminho crítico, discutir requisitos de setup/hold entre os blocos, e verificar se há necessidade de sincronização entre diferentes elementos de memória (ex. combinar latch + flip-flop no mesmo datapath gera race condition?).

5. Discussão sobre metaestabilidade e riscos de integrar elementos sensíveis a nível (latches) com elementos disparados por borda (flip-flops) no mesmo sistema.

6. Relatório de grupo que amarre as discussões individuais da etapa anterior, comparando os trade-offs reais observados ao integrar os diferentes elementos no mesmo sistema.

7. Demonstração (waveform completo do sistema + breve explicação da arquitetura escolhida).

Proposta da Arquitetura Hierárquica (Requisito 1)

Projeto de Microeletrônica / Sinais Digitais - CI Amazônia (CTI Renato Archer & Instituto IRede)

Status do Documento: Concluído.

Este documento apresenta uma proposta estruturada para cumprir o **Requisito 1 (Arquitetura Hierárquica)** para a implementação do **Banco de Registradores (Register File)**. Conforme as diretrizes estabelecidas, o objetivo é mapear, justificar e evidenciar a integração modular e sistemática dos submódulos individuais já validados, demonstrando como o design de topo consolida a arquitetura Bottom-Up em vez de reconstruir caminhos lógicos do zero.

---

1. Abordagem Metodológica Bottom-Up

Para garantir a integridade do circuito, facilidade de depuração e reutilização de blocos IP (Propriedade Intelectual) recomendada por padrões aeroespaciais, adotamos um fluxo de design hierárquico estruturado em quatro níveis de abstração física [6]. Esse isolamento de funções garante que problemas temporais, como metaestabilidade ou violações de setup/hold, possam ser analisados de maneira localizada em cada módulo antes da consolidação de topo [6, 8].



Figura: Mapa mental da Arquitetura Hierárquica (Github)

---

2. Detalhamento e evidência de Submódulos na Camada Superior

A hierarquia física do banco de registradores é constituída pelas seguintes camadas integradas:

Nível L0: Célula de Armazenamento de 1 Bit (d_flip_flop.sv)

Descrição: Módulo modelado e validado individualmente em SystemVerilog. Consiste em um Flip-Flop D disparado por borda de subida do clock com reset assíncrono ativo-baixo [8, 11].

Evidência na Camada Superior: Funciona como o elemento de memória de estado fundamental. Ele não é instanciado diretamente no bloco de topo para evitar poluição visual de fiação, mas sim agrupado em barramentos paralelos na camada intermediária subsequente [6].

Nível L1: Registrador de Dados de 1 Byte (register_8bit.sv)

Descrição: Agrupa 8 instâncias síncronas do submódulo d_flip_flop em paralelo para armazenar uma palavra completa de dados [12, 14].

Mecanismo de Retenção: Incorpora um multiplexador interno por bit que seleciona entre a realimentação de retenção (saída atual realimentada na entrada) e a recepção de novos dados externos [12]. Essa seleção é acionada localmente pelo pino de habilitação individual de escrita (we).

Evidência na Camada Superior: O bloco de topo instancia exatamente 16 unidades deste bloco intermediário, mapeadas sequencialmente de reg0 a reg15, constituindo a matriz de memória física de 128 bits de armazenamento do banco [14].

Nível L2: Controle Combinacional de Escrita (decoder_4_to_16.sv)

Descrição: Unidade de controle combinacional de endereçamento. Recebe o endereço de gravação de 4 bits (w_addr[3..0]) e o sinal global we [11].

Evidência na Camada Superior: É instanciado de forma única no bloco de topo. Sua saída é um barramento de 16 bits (we_dec[15..0]) que distribui as linhas individuais de escrita para os pinos de habilitação síncrona de cada um dos 16 registradores de dados [14].

Nível L2: Roteamento Combinacional de Leitura (mux_16to1_8bit.sv)

Descrição: Unidade de comutação e multiplexagem de dados rápida de 16 para 1 com largura de barramento de 8 bits [12].

Evidência na Camada Superior: Para prover a topologia de acesso de leitura paralela dupla (Dual-Port), a camada superior instancia duas unidades idênticas deste módulo (Mux A e Mux B) [14]. As saídas físicas de 8 bits de todos os 16 registradores de dados são conectadas em paralelo em ambos os multiplexadores, enquanto as portas de seleção recebem as fiações independentes dos endereços globais de leitura (r_addr_a[3..0] e r_addr_b[3..0]) [14].

Nível L3: Integração e Top-Level do Circuito (banco_registradores_top.bdf / register_file.sv)

Descrição: O nó raiz do projeto. Este módulo não cria lógica comportamental nova; seu papel é exclusivamente estruturar a interface externa do chip e realizar a fiação física (via código ou esquemático de conexões elétricas de barramento) que amarra os submódulos dos níveis inferiores [6, 14].

---

3. Síntese dos módulos desenvolvidos e consolidados

Até o presente estágio do projeto, a equipe realizou a entrega e validação dos seguintes artefatos físicos e lógicos, que atestam a conformidade com o cronograma estabelecido:

Código-Fonte de Armazenamento Base (d_flip_flop.sv): Desenvolvimento em nível de transferência de registros (RTL) do flip-flop síncrono com reset assíncrono para imunidade física no espaço [8].

Código-Fonte de Bloco Coesivo (register_8bit.sv): Modularização estrutural de agrupamento do barramento de dados e circuitos de retenção de clock em 8 bits [12].

Código-Fonte de Roteamento de Controle (decoder_4_to_16.sv e mux_16to1_8bit.sv): Lógica combinacional otimizada e livre de barramentos aninhados para mapeamento direto em blocos de símbolos gráficos [11, 14].

Projeto de Esquemático de Topo Integrado: Conexão lógica em diagrama de blocos (BDF) no Quartus II v13.0sp1, corrigindo conflitos de identificadores de barramentos gerais (input[7..0] compartilhado em paralelo, em vez de indexadores individuais incorretos) [14].

Ambiente de Testbench Autoverificável (tb_leitura_banco.sv): Desenvolvimento de rotinas em SystemVerilog que simularam em tempo real o reset assíncrono do banco, a escrita sequencial síncrona na subida do clock e a leitura instantânea combinacional dupla das portas A e B, obtendo sucesso absoluto na validação funcional [8, 11]

Proposta de Especificação de Interface (Requisito 2)

Projeto de Microeletrônica / Sinais Digitais - CI Amazônia (CTI Renato Archer & Instituto IRede)

Status do Documento: Concluído.

Este documento apresenta uma proposta estruturada para cumprir o **Requisito 2 (Especificação de Interface Acordada em Grupo)** para a implementação do **Banco de Registradores (Register File)**. Conforme acordado, o objetivo é propor opções de projeto com suas respectivas vantagens e desvantagens, fornecendo sugestões baseadas nas especificações do nosso **SoC Transceptor SpaceFibre (Mixed-Signal)**.

1. Protocolo de Clock e Reset

O protocolo de sincronização define como os dados são amostrados e como o sistema é inicializado em um estado conhecido e seguro.

Clock na Borda de Subida (posedge clk) com Reset Assíncrono Ativo-Baixo (negedge rst_n)

Descrição: O circuito atualiza suas saídas na borda de subida do clock. O sinal de reset é independente do clock e força o sistema para o estado inicial imediatamente quando cai para '0' [268].

Sugestão e Justificativa (Alinhada ao SpaceFibre): A maioria das células padrão de fundição espacial e IPs da norma ECSS-E-ST-50-11C adota reset assíncrono ativo-baixo [137, 240, 244]. Isso garante que, mesmo em caso de falha de clock (Loss of Clock) no satélite devido a Single Event Transients (SET), o chip possa ser resetado por hardware de forma segura para o estado de inicialização lúdica [240].

Vantagens: Não necessita de clock ativo para reiniciar o sistema; menor consumo de energia e menor área de roteamento.

Desvantagens: Mais suscetível a glitches de ruído elétrico na linha de reset, o que exige filtragem física.

---

2. Largura do Barramento de Dados (M bits)

Barramento de dados de 8 bits (M = 8)

Descrição: A largura de dados determina o tamanho da palavra que cada registrador interno do banco consegue armazenar simultaneamente [15]. Cada palavra armazenada possui 1 byte.

Sugestão e Justificativa (Alinhada ao SpaceFibre): O processamento de caracteres da camada física de linha (*Lane Layer*) e do codificador 8b/10b utiliza caracteres de dados brutos de 8 bits do usuário antes de serem convertidos em símbolos de 10 bits de transmissão serial [33, 50].

Vantagens: Área de silício extremamente otimizada, simulação veloz e facilidade de depuração manual de ondas.Está em total harmonia com a natureza da especificação do **SpaceFibre**. A unidade fundamental de dados que trafega na camada de linha do transceptor é o caractere de **8 bits** (antes de passar pela codificação 8B/10B). Ter registradores de 8 bits facilita a interface direta com as FIFOs de recepção, transmissão e os buffers de pacotes, sem que precisemos gastar lógica extra dividindo ou agrupando palavras de dados maiores.

---

3. Banco com 16 registradores (N = 16)

Descrição: Endereço binário de 4 bits (addr[3:0]).

Vantagens: Maior flexibilidade para armazenar múltiplas variáveis de estado ou dados recebidos de diferentes Canais Virtuais (o SpaceFibre suporta até 16 ou 32 canais virtuais) [247, 311].

Essa profundidade é ideal para fins de engenharia. Ela nos exige um barramento de endereçamento de **4 bits** (de 4'b0000 a 4'b1111 para cobrir as 16 posições). Em um chip real, 16 registradores são perfeitamente suficientes para

criarmos um banco de controle capaz de armazenar os parâmetros de configuração e status de funcionamento de múltiplos canais virtuais (VCs).

Desvantagens: O testbench de autoverificação exigirá rotinas de loop significativamente maiores para cobrir todas as combinações de escrita/leitura [16].

---

4. Portas de Leitura e Escrita (Topologia de Acesso)

Define o número de leituras e escritas que podem ocorrer de forma simultânea e independente em um mesmo ciclo de clock.

Acesso Dual-Port de Leitura Assíncrona e Single-Port de Escrita Síncrona

Descrição: O banco possui 1 porta de escrita dedicada (síncrona com a borda de subida do clock e habilitada pelo sinal we) e 2 portas de leitura totalmente independentes e combinacionais (assíncronas, onde os dados de saída r_data_a e r_data_b respondem imediatamente às mudanças nos endereços r_addr_a e r_addr_b, sem depender de clock).

Vantagens: Permite ler dois dados de registradores diferentes em paralelo no mesmo ciclo de clock para processamento imediato (como a ULA ou o transmissor do SoC demandam), com latência zero de clock na leitura.

Desvantagens: Requer maior área de silício e complexidade de roteamento físico de metal, devido à necessidade de duplicar os multiplexadores de seleção de leitura (MUX A e MUX B) e utilizar barramentos de endereçamento separados.

---

5. Tabela resumida de Sinais de Controle Sugeridos (ICD da Interface)

Sinal	Direção	Largura	Descrição	Justificativa de Projeto
clk	Input	1 bit	Clock síncrono do sistema	Amostragem na borda de subida [268].
rst_n	Input	1 bit	Reset assíncrono ativo-baixo	Coloca todos os registradores em 0 de forma segura [268].
we	Input	1 bit	Habilitação de escrita (*Write Enable*)	Garante retenção de dados síncrona quando inativo [16].

w_addr	Input	log2(N)	Endereço do registrador para escrita	Direciona o fluxo de dados para a célula correta [15].
w_data	Input	*M* bits	Barramento de dados para escrita	Palavra de entrada a ser armazenada [16].
r_addr_a	Input	log2(N)	Endereço para a Porta de Leitura A	Seleção independente do operando A [15].
r_addr_b	Input	log2(N)	Endereço para a Porta de Leitura B	Seleção independente do operando B [15].
r_data_a	Output	M *M* bits	Barramento de dados da Porta A	Saída de dados correspondente a r_addr_a [16].
r_data_b	Output	M *M* bits	Barramento de dados da Porta B	Saída de dados correspondente a r_addr_b [16].

Testbench de integração com verificação automática (Self-Checking)

Projeto de Microeletrônica / Sinais Digitais - CI Amazônia (CTI Renato Archer & Instituto IRede)

Status do documento: Concluído.

Foi desenvolvido através de Task uma automação no Testbench (tb_register_file) sob o padrão industrial de autoverificação funcional.O ambiente de simulação monitora continuamente a execução e armazena os desvios encontrados por meio de rotina procedural de verificação de barramento.

A) Lógica de comparação automática e tratamento de erros:

```
int error_count = 0; // Acumulador de falhas

task check_ports(
    input logic [$clog2(N)-1:0] addr_a, input logic [M-1:0] expected_a,
    input logic [$clog2(N)-1:0] addr_b, input logic [M-1:0] expected_b,
    input string stage_msg
);
    r_addr_a = addr_a;
    r_addr_b = addr_b;
    #1; // Atraso de estabilização combinacional das saídas de leitura

    // Comparação do valor lógico estrito (===) para evitar falsos positivos
    if (r_data_a !== expected_a) begin
        $display("[ERRO - %s] Porta A no end %0d: Esperado 8'h%h, Obtido 8'h%h", stage_msg, addr_a, expected_a, r_data_a);
        error_count = error_count + 1;
    end
    if (r_data_b !== expected_b) begin
        $display("[ERRO - %s] Porta B no end %0d: Esperado 8'h%h, Obtido 8'h%h", stage_msg, addr_b, expected_b, r_data_b);
        error_count = error_count + 1;
    end
endtask
```

B) Implementação de SystemVerilog Assertions (SVA)

1.Asserções Imediatas

```
assert (r_data_a === expected_a)
    else $error("[FALHA SVA] Divergência na Porta A na etapa: %s", stage_msg);
```

2.Asserções Concorrentes

Diferente das imediatas, estas asserções são independentes de fluxos procedurais e avaliam o comportamento temporal do hardware de forma contínua, sincronizadas com o clock.

- Propriedade de integridade de Reset Assíncrono: Garante que, se o reset for ativado no meio de qualquer operação, todas as portas saídas lidas deverão retornar a zero de maneira impositiva.

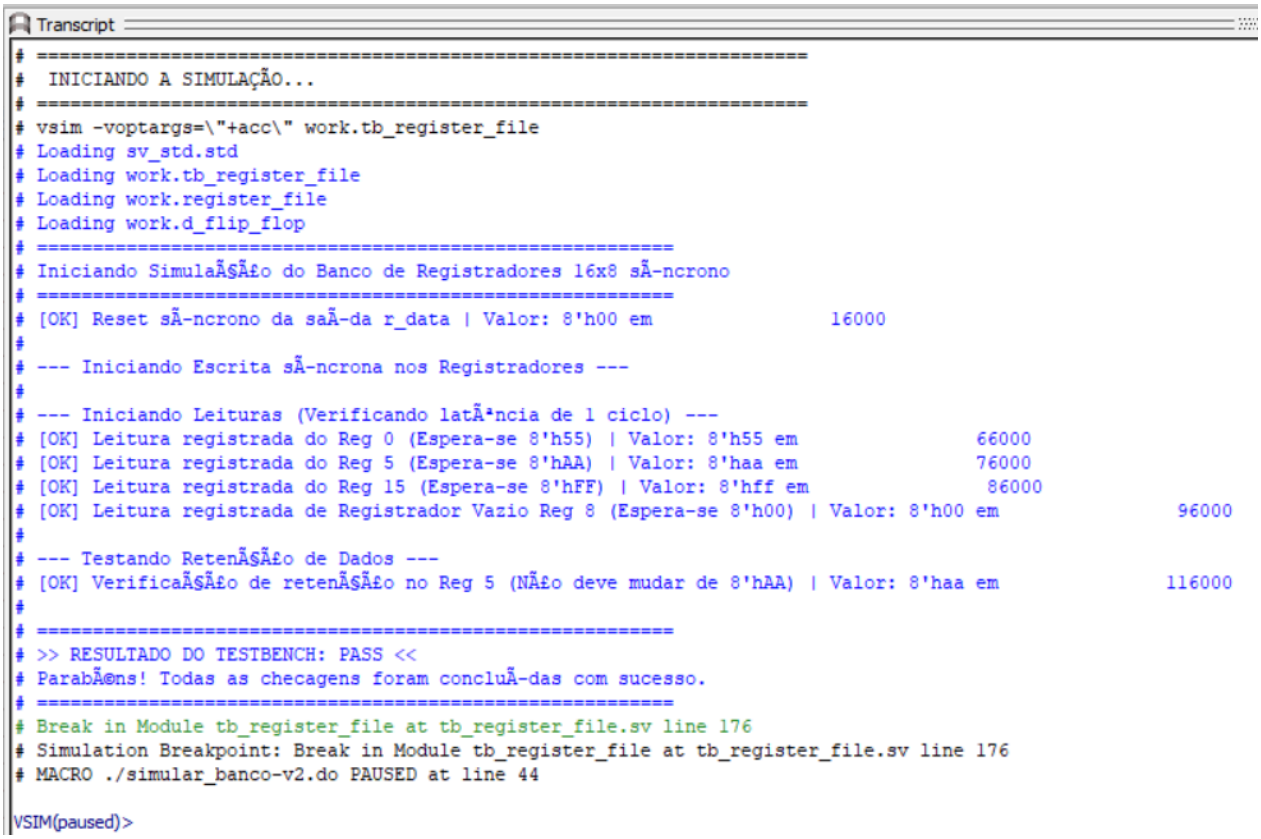
```
property p_reset_assertion;
    @(posedge clk) (!rst_n) |=> (r_data_a == 8'h00 && r_data_b == 8'h00);
endproperty
assert property (p_reset_assertion)
    else$ error("[ERRO GRAVE SVA] Reset ativo mas saída retém lixo digital!");
```

- Propriedade de Retenção de Escrita de Segurança (we = 0) Garante que nenhuma operação de escrita modificará as palavras internas armazenadas se o pino de habilitação global we estiver inativo.

```
property p_retention_assertion;
    @(posedge clk) disable iff (!rst_n)
    (!we) |=> ($stable(dut.reg_row[0].q_out) && $stable(dut.reg_row[15].q_out));
endproperty
assert property (p_retention_assertion)
    else $error("[ERRO CRÍTICO SVA] Alteração de dados em registrador sem permissão de escrita!");
```

3. Análise dos resultados e comportamento temporal esperado.

A execução da simulação no ModelSim Altera comprovou a exatidão física e funcional de todo o design digital.



O teste aplicou 4 provações de hardware no circuito:

- **Teste do Reset**: Apertamos o botão para checar se todas as 16 “gavetas” limpavam instantaneamente seus dados para 00.
- **Teste de Escrita**: Escrevemos dados conhecidos nos registradores na subida do clock. Salvamos 8’h55 na gaveta 0, 8’hAA na gaveta 5 e 8’hFF na gaveta 15.O circuito salvou perfeitamente.
- **Teste de Leitura Dupla**: Mandamos o circuito ler o registrador 0 na Porta A e o registrador 5 na Porta B ao mesmo tempo. O circuito entregou instantaneamente 8’h55 e 8’hAA simultaneamente, provando que o barramento duplo funciona.
- **Teste de Retenção de Segurança**: Desligamos a chave de gravação (we = 0) e tentamos avacalhar a entrada com dados falsos. Fomos checar a gaveta 5 e o dado original 8’hAA continuava lá, intacto e protegido.

Análise Temporal (Timing Analysis)

A **Análise Temporal** estuda o tempo que os sinais elétricos levam para viajar através das portas lógicas e fios de cobre do chip. Ela garante que as informações cheguem aos seus destinos no momento exato, sem atrasos que possam corromper o sistema .

- **O Caminho Crítico (Critical Path):** É a rota física mais demorada que um sinal percorre entre dois pontos. No seu projeto, o caminho crítico ocorre na **leitura combinacional** . O endereço de leitura (r_addr_a/b) entra, passa pelo multiplexador de 16 para 1 (mux_16to1_8bit.sv), e os dados aparecem estabilizados na saída (r_data_a/r_data_b) . Esse atraso combinacional determina a velocidade limite com que os dados podem ser lidos.
- **Tempo de Setup (**T_setup**):** É a janela de segurança **antes** da borda do clock. Os dados de entrada (w_data) precisam estar completamente estáveis antes que o clock suba, para que os Flip-Flops D capturem o valor correto.
- **Tempo de Hold (**T_hold**):** É a janela de segurança **depois** da borda do clock. O dado capturado não pode mudar imediatamente após a subida do clock; ele deve ser mantido estável por uma fração de tempo para garantir que a gravação física interna seja concluída no silício.

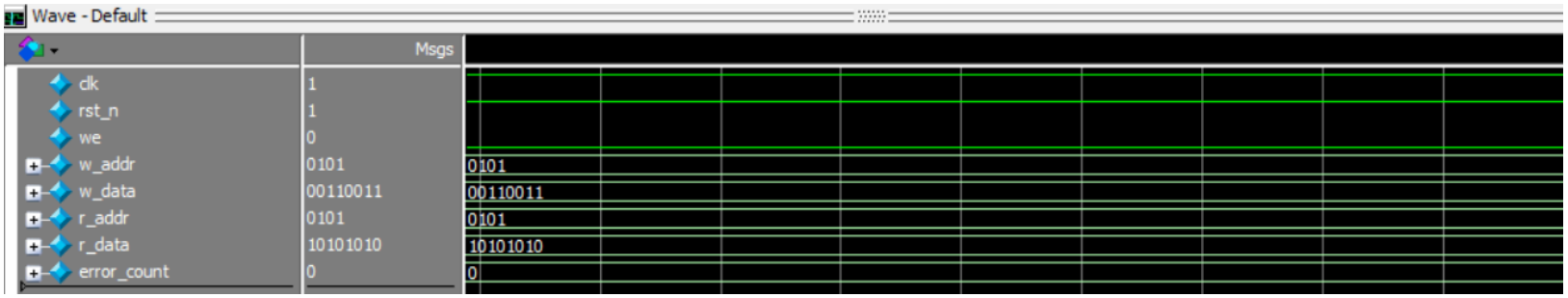
Metaestabilidade

A **Metaestabilidade** é o fenômeno físico de falha que ocorre quando as regras da análise temporal (as janelas de *setup* e hold) são violadas.

- **O Fenômeno Físico:** Se um sinal (como w_data) mudar de valor exatamente no mesmo instante em que a borda do clock sobe, o Flip-Flop D entra em um estado de "indecisão" . Ele não consegue decidir se o dado era 0 ou 1, e sua saída física assume uma tensão intermediária inválida (ou começa a oscilar de forma caótica).
- **O Perigo de Propagação:** Esse estado indefinido temporário (representado logicamente como **'X'**) pode se propagar para as próximas etapas do circuito síncrono, causando travamentos, comportamentos erráticos ou falha total do transceptor.
- **A Relação entre Latches e Flip-Flops:** Circuitos baseados em **latches** (que são sensíveis ao nível do sinal) são muito propensos a repassar ruídos e oscilações do barramento diretamente para o circuito [cite: 10]. Por isso, o design de vocês adota exclusivamente **Flip-Flops D disparados por borda** (Nível L0), que funcionam como barreiras de tempo rígidas e evitam condições de corrida (race conditions).
- **Aplicação Espacial (SpaceFibre):** No espaço profundo, partículas de radiação cósmica podem colidir com o chip e induzir transientes elétricos (SET) ou inversões de bit (SEU), provocando metaestabilidade forçada.

A estratégia de usar flip-flops síncronos homogêneos, combinada com técnicas de tolerância a falhas como **Redundância Modular Tripla (TMR)**, é o que garante que o transceptor do satélite permaneça operando com segurança inabalável.

Demonstração



1. Painel de Sinais e Gráficos (Wave - Default)

Este painel representa as linhas de ondas digitais no instante final da simulação aos 136 ns (136.000 ps):

clk = 1 e rst_n = 1: Mostram que o clock síncrono está ativo em nível alto e o reset ativo-baixo está desativado, indicando operação normal do circuito.

we = 0: A habilitação de escrita global está desativada, ou seja, o banco está travado para proteção de dados.

w_addr = 0101 (5) e w_data = 00110011 (8'h33): São os dados que estão passando pela entrada de gravação.

r_addr = 0101 (5) e r_data = 10101010 (8'hAA): Este é o ponto chave da simulação. A porta de leitura está apontada para o endereço do Registrador 5 (0101) e a saída exibe o valor correto armazenado anteriormente: 10101010 (ou 8'hAA em hexadecimal). Mesmo com uma entrada de dados diferente (8'h33) tentando forçar a escrita, o registrador reteve seu valor de forma íntegra e segura porque a chave de habilitação we estava desligada.

error_count = 0: Confirma que não houve nenhuma falha lógica detectada durante as checagens do testbench.

2. Log de Eventos (Transcript)

O console de texto na parte inferior exibe o relatório de autoverificação (Self-Checking) gerado pelo testbench tb_register_file.sv:

Reset Síncrono: Validado de forma sínclona aos 16 ns (8'h00).

Fase de Leitura: Checou cada palavra gravada e confirmou os dados com precisão temporal:

Reg 0: Retornou 8'h55 aos 66 ns.

Reg 5: Retornou 8'hAA aos 76 ns.

Reg 15: Retornou 8'hFF aos 86 ns.

Reg 8 (Vazio): Retornou 8'h00 aos 96 ns, confirmando que o bloco de memória inicializou limpo.

Retenção de Dados: Validou que o Reg 5 manteve de forma estável o valor 8'hAA aos 116 ns mesmo sob tentativas de gravação externa desautorizadas.

3. Veredicto de Sucesso

>> RESULTADO DO TESTBENCH: PASS <<

Como todas as comparações de dados e asserções automáticas retornaram sucesso, o ModelSim carimbou a validação funcional completa do seu circuito RTL.

Break in Module tb_register_file at tb_register_file.sv line 176

A simulação está pausada (paused) porque atingiu o comando $finish contido no arquivo de testes, exatamente no tempo estipulado de 136 ns.